CS 6650 Final Mastery — Deliverable Q&A

Zongwang Wang · ChaosArena v1-album-store · 190/190

---

1. Roughly how many submissions did it take before you passed all critical scenarios (S1–S5), and what was the most common failure?

All five critical scenarios passed on the **first ChaosArena submission**. Before submitting, every endpoint was smoke-tested manually with `curl`: health check, album create/get/list, photo upload with status polling, delete with the 404 verification, concurrent seq assignment.

The most common failure caught during local testing was a `seq=0` bug in the GET photo response when status was `completed`. The worker goroutine was updating Redis without the `seq` field because `PhotoJob` was missing a `Seq int64` field. The fix was adding it and passing it through. Had this reached ChaosArena, S3 and S10 would have failed.

A second correctness bug — caught only after the first ChaosArena runs — was an S9 race condition: the worker could write `status=completed` to Redis *after* the DELETE handler evicted the cache, making a deleted photo appear as completed. Fixed by having `UpdateStatus` return `RowsAffected() > 0` and skipping the Redis write if 0 rows were affected.

---

2. Where are your photo files stored, and why did you pick that over other options?

Photo files are stored in **Amazon S3** (`album-store-330214284546-us-west-2`, region `us-west-2`). The bucket has a public-read policy so objects are accessible at `https://<bucket>.s3.us-west-2.amazonaws.com/<key>` with no expiry.

Why S3 over alternatives:

Option	Reason rejected
Local disk on EC2	Lost on instance replacement; requires a file server for the public URL; shared storage across 4 nodes is complex
EFS (shared filesystem)	Adds cost and complexity; S3 is already optimal
RDS / BLOB	Saturates the database; no HTTP URL without a download endpoint
Third-party CDN	Unnecessary; S3 in the same region as EC2 already gives 100+ Gbps internal bandwidth

With EC2 and S3 both in `us-west-2`, uploads happen over AWS internal network — extremely fast. The `url` field requirement ("must return HTTP 200 when fetched") is trivially satisfied by S3's public-read policy.

---

3. Describe your deployment setup — how many instances, what cloud services, and how they connect to each other.

Final setup for 190/190:

- **4× EC2 c5n.large** — run the Go HTTP server on port 8080. All behind an AWS ALB.
- **AWS ALB** — internet-facing, routes port 80 → port 8080 on the target group.

- **ElastiCache cache.t3.micro (Redis 7)** — shared across all 4 nodes via a single managed endpoint. Used for atomic seq INCR and read caching.
- **RDS PostgreSQL 17 db.t3.medium** — durable storage for albums and photos. In the same VPC, accessible by all 4 nodes.
- **S3 bucket** — photo file storage. Accessed via IAM instance role (no hardcoded credentials). Public-read for fetchable URLs.

All EC2 instances, RDS, and ElastiCache share the same VPC security group (`sg-0f6369c91322c7ba7`). Inbound rules allow: port 22 (SSH), port 80 (ALB), port 8080 (HTTP), port 6379 (Redis within SG), port 5432 (PostgreSQL within SG).

Deployment is done via SSM Run Command: binary is built locally, uploaded to S3, then pulled down and restarted on all 4 EC2 nodes in parallel.

---

4. Did you use a reverse proxy or load balancer? If so, what role does it play in your architecture?

Yes — an **AWS Application Load Balancer (ALB)**.

Role: the ALB distributes incoming ChaosArena requests across all 4 c5n.large nodes using round-robin routing. ChaosArena submits one `base_url` (the ALB DNS name) and all traffic flows through it. The ALB runs health checks on `/health` every 10 seconds and routes only to healthy instances.

The ALB is critical for the load test scores:

- **4× the total network bandwidth**: each node has 25 Gbps dedicated; 4 nodes = 100 Gbps total capacity for S3 uploads.
- **4× the concurrency**: S12 (concurrent photo uploads) distributes across 4 nodes, each running its own goroutine pool with the tiered semaphore.
- **Reliability**: if one node has a GC pause or is processing a large upload, the ALB routes new requests to other nodes.

Without the ALB + multiple nodes, a single c5n.large scored 181/190 — the extra bandwidth from 4 nodes was needed to hit 190.

---

5. How does your background worker get notified that there's a new photo to process? Did you use a queue, polling, or something else?

A **Go goroutine with a tiered semaphore** — no external queue, no polling.

```go
// After sending 202:
go func() {
    defer file.Close()
    defer mf.RemoveAll()

    // Tiered semaphore: pick slot based on file size
    if fileSize >= 10*1024*1024 {  // ≥ 10 MB
        largeSem <- struct{}{}      // 8 slots per node
        defer func() { <-largeSem }()
    } else {
        smallSem <- struct{}{}      // 64 slots per node
        defer func() { <-smallSem }()
    }
```

```
    // S3 upload happens here
}()
```

The handler buffers the file (ParseMultipartForm), sends 202, then launches this goroutine. The goroutine holds a semaphore slot during the S3 upload and releases it when done. No channel handoff, no queue.

Why not a persistent worker pool: A fixed-size pool adds queuing overhead and a channel. Direct goroutines are simpler, equally controlled by the semaphore, and spawn/die immediately — no idle goroutines consuming memory.

Why not an external queue (SQS, Redis list): Adds latency (network round-trip per job), complexity, and cost. An in-process goroutine has nanosecond launch latency. The trade-off is jobs are lost on process crash, but for a load test this is acceptable.

---

6. The spec requires that `seq` is assigned in the POST handler, not the background worker. Why does that matter, and how did you ensure correctness under concurrent uploads to the same album?

Why it matters: If seq were assigned in the worker, the 202 response couldn't include it (the worker runs asynchronously). More critically, the order workers process jobs is nondeterministic — two concurrent uploads might receive seq values in the wrong order relative to submission time. Assigning in the handler ensures the client receives the correct seq in the 202 body and that assignment order reflects submission order.

How correctness is ensured: Redis `INCR` via ElastiCache:

```go
func (c *Cache) NextSeq(ctx context.Context, albumID string) (int64, error) {
    return c.client.Incr(ctx, "album:"+albumID+":seq").Result()
}
```

Redis processes commands in a single thread. `INCR` is atomic — two goroutines calling `INCR album:X:seq` simultaneously always receive different values. With 4 server nodes all pointing to the same ElastiCache endpoint, global atomicity is guaranteed across the entire cluster.

Alternatives considered and rejected:

- **`SELECT MAX(seq) + 1` in DB:** Two concurrent transactions can read the same max and both try to insert the same seq — unique constraint violation or silent duplicate.
- **Per-instance `atomic.Int64`:** Correct on a single instance, but 4 nodes each have their own counter → all start at 0 → duplicates.
- **DB `UPDATE ... RETURNING`:** Correct but adds a ~3ms DB round-trip per upload. Redis INCR is ~0.1ms.

---

7. What happens in your system if the worker crashes or fails halfway through processing a photo?

Several safeguards:

S3 upload failure (network error, S3 rate limit): The goroutine catches the error and calls `UpdateStatus(ctx, photoID, "failed", "")`. The photo transitions to `status='failed'`. ChaosArena accepts `failed` as a valid terminal state. If the photo was already deleted (delete-during-upload race), `UpdateStatus` returns 0 rows affected — the goroutine skips the cache write and returns cleanly.

Delete during upload (S7/S8/S9 trap): The goroutine checks `IsDeleted` before starting the S3 upload (fast path), and `UpdateStatus` itself is `WHERE status != 'deleted'` — so even if the delete races between those two checks, 0 rows are affected and the goroutine cleans up the just-uploaded S3 object.

Process crash mid-upload: The file handle (backed by temp file or memory) is garbage-collected when the goroutine exits. The photo row remains `status='processing'` in the database permanently — effectively a stuck job. ChaosArena's 30-second polling timeout would catch this as a failure.

What would fix this in a real system: A persistent queue (SQS with visibility timeout) would allow re-queueing if the consumer crashes. For this assignment the goroutine approach was sufficient.

---

8. What does your database schema look like? What tables or collections did you create and why?

Two tables:

```
CREATE TABLE IF NOT EXISTS albums (
    album_id    UUID PRIMARY KEY,
    title       TEXT NOT NULL,
    description TEXT NOT NULL,
    owner       TEXT NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS photos (
    photo_id   UUID PRIMARY KEY,
    album_id   UUID NOT NULL,
    seq        BIGINT NOT NULL,
    status     TEXT NOT NULL DEFAULT 'processing',
    url        TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (album_id, seq)
);
```

Design decisions:

- `album_id` in `photos` has **no foreign key** to `albums`. FK constraints acquire a share lock on the parent row during child inserts, which creates contention under S12 (many concurrent photo inserts for the same album). The application validates album existence before inserting.
- `status` is `TEXT` with values `'processing' | 'completed' | 'failed' | 'deleted'` rather than PostgreSQL ENUM. ENUMs require a schema migration to add values; TEXT is flexible.
- `url` is nullable — `NULL` means the photo has not yet been uploaded to S3.
- `UNIQUE (album_id, seq)` is a last-resort safety net. Redis INCR prevents duplicates in practice.
- No `album_seq` table — seq counters live in Redis, not the DB.

---

9. Did you add any indexes to your database? If so, on which columns and why?

One explicit index beyond the primary keys:

```
CREATE INDEX IF NOT EXISTS idx_photos_album_id ON photos(album_id);
```

Used by the seq warm-up query at startup:

```
SELECT album_id, MAX(seq) FROM photos WHERE status != 'deleted' GROUP BY album_id
```

Without the index, this is a full table scan over potentially thousands of photos accumulated across test runs.

The `UNIQUE (album_id, seq)` constraint also creates an implicit index on `(album_id, seq)`.

For `GET /albums` with 260,000+ accumulated albums, the key insight was using **UUID-typed keyset pagination** instead of text-cast:

```
-- Uses primary key index (fast):
WHERE album_id > $1::uuid ORDER BY album_id LIMIT 500

-- Bypasses index due to type cast (30+ second full scan):
WHERE album_id::text > $1
```

---

10. Which load testing scenario was the hardest for you, and what bottleneck did you discover?

S15 (Large Payload Upload, 20 pts) — exposed multiple distinct bottlenecks:

Bottleneck 1 — t3.large burst credit exhaustion. After 3–4 consecutive load test runs, the t3.large's CPU/network credits dropped from 288 to ~13. At that point the instance throttled network throughput from 5 Gbps (burst) to ~256 Mbps (baseline). S15 scored 0 on one run because error rate exceeded 10% (uploads timing out). CloudWatch confirmed credit exhaustion was the cause.

Bottleneck 2 — HTTP/1.1 streaming limitation. Attempted to send 202 before reading the full request body (streaming client→server→S3 in parallel). Failed because HTTP/1.1 clients close the TCP send side after receiving a complete 202 response, invalidating `r.Body`. Multiple approaches tried: io.Pipe with handler goroutine reading, sub-goroutine reading — all failed with "http: invalid Read on closed Body."

Bottleneck 3 — Bandwidth fragmentation. With a single instance and 32 concurrent large uploads, each upload got 1/32 of bandwidth. With 4 nodes × 25 Gbps = 100 Gbps total and a tiered semaphore limiting large uploads to 8 per node (32 total), each upload gets ~3 Gbps → 200 MB in ~0.5 seconds.

Final S15 score: 20/20 — accept_p95=4,098ms, complete_p95=6,188ms.

---

11. What was the single most impactful change you made to improve your load test scores?

Switching from 1× t3.large to 4× c5n.large behind an ALB.

Before: 179/190 (best on single instance) After: 190/190

The c5n.large has **25 Gbps dedicated network** (no burst credits, no throttle under sustained load). With 4 nodes, total capacity is 100 Gbps. This directly determined S12 (15/15, p95=1,368ms vs 2,615ms before) and S15 (20/20, complete_p95=6,188ms vs 10,641ms before).

The second most impactful change was the **tiered semaphore** (64 slots for < 10 MB files, 8 slots for ≥ 10 MB). Without it, either small files (S12) would be throttled unnecessarily, or large files (S15) would split bandwidth across too many concurrent uploads.

The third most impactful was the **StreamList with UUID index** — fixing `GET /albums` from a 30-second timeout (260K rows, text-cast sequential scan) to 1.27 seconds (UUID index scan). Without this fix, S5 always failed, blocking all load tests.

---

12. How did you handle concurrent writes — for example, many album creates or photo uploads happening at the same time?

Album creates (S11 — concurrent PUT /albums): PostgreSQL's `INSERT ... ON CONFLICT (album_id) DO UPDATE` handles concurrent creates atomically — exactly one insert wins, the rest become updates. With

`pgxpool(25 connections per node × 4 nodes = 100 total)`, requests don't queue waiting for connections. db.t3.medium handles the concurrency without connection exhaustion.

Photo seq assignment (S10/S12 — concurrent uploads to same album): Redis `INCR album:{id}:seq` via ElastiCache. Globally atomic across all 4 server nodes. No application-level locking needed.

Photo inserts (S12/S14 — concurrent photo rows): The `photos` table has no FK to `albums` (avoids FK lock contention). Each insert is independent. The `UNIQUE (album_id, seq)` constraint rejects any theoretical duplicate, but Redis INCR prevents them from occurring.

S3 upload concurrency: Tiered semaphore caps concurrent uploads at 64 (small) or 8 (large) per node. This prevents bandwidth fragmentation while still saturating available throughput.

---

13. Describe a specific bug you ran into and how you diagnosed it using the ChaosArena event logs or your own logs.

Bug 1: `seq=0` in GET photo response when status=completed

During local smoke testing before the first ChaosArena submission:

```
POST /albums/:id/photos → {"photo_id":"abc...","seq":1,"status":"processing"} ✓
GET  /albums/:id/photos/abc... → {"seq":0,"status":"completed",...}            ✗
```

The 202 response had `seq=1` but the completed GET response had `seq=0`. Traced through the code: the POST handler assigned `seq=1` via Redis INCR and seeded Redis with the correct value. The worker goroutine uploaded to S3, then called `cache.SetPhotoStatus()` with a `PhotoStatus` struct — which omitted the `Seq` field. Go zero-initializes `int64` to 0, overwriting the correct value.

Fix: Added `Seq int64` to `PhotoJob` and used it in the worker's cache update.

Bug 2: S9 race — deleted photo reappearing as completed

After the first runs showed S9 occasionally failing, the race was identified: DELETE marks `status='deleted'` in DB, evicts Redis cache → Worker finishes S3 upload → Worker's `IsDeleted()` check passes (race window was between delete and check) → Worker calls `UpdateStatus(DB, "completed")` → 0 rows affected (`WHERE status != 'deleted'`) → Worker then calls `cache.SetPhotoStatus("completed")` → Redis now shows completed again, even though DB shows deleted.

Fix: `UpdateStatus` now returns `(bool, error)` — the bool is `RowsAffected() > 0`. Worker skips the Redis cache write if the DB row was not updated (already deleted). After this fix, S9 never failed again.

Bug 3: GET /albums returning empty response (S5 failing)

After multiple load test runs accumulated 260,000+ albums in the database, S5 started failing with "empty reply from server." Server logs showed the `/albums` endpoint hanging for 30+ seconds.

Diagnosed by querying the DB directly: `SELECT COUNT(*) FROM albums` returned 260,686. The keyset pagination query used `WHERE album_id::text > $1` which forces a cast of all UUID values to text — bypassing the UUID primary key index and triggering a full table scan on 260K rows.

Fix: Changed to `WHERE album_id > $1::uuid` (UUID-typed comparison, uses the primary key index) and switched from buffering all albums to streaming them via `json.NewEncoder(w)`. Result: 260K albums in 1.27 seconds.

---

14. How did you test your service locally before submitting to ChaosArena?

"Locally" meant testing against the deployed EC2 servers (since the service requires RDS, Redis, and S3). Testing was done with `curl` commands and Python one-liners.

Smoke test sequence (run before every submission):

```
BASE="http://<ALB-or-EC2-IP>:8080"
AID=$(python3 -c "import uuid; print(str(uuid.uuid4()))")

# S1/S6: health — must be exactly {"status":"ok"}
curl -s "$BASE/health"

# S2: album CRUD + idempotency
curl -s -X PUT "$BASE/albums/$AID" -H "Content-Type: application/json" \
  -d '{"album_id":"'$AID'","title":"T","description":"D","owner":"o@o.com"}'
curl -s "$BASE/albums/$AID"        # must echo back all 4 fields
curl -s -X PUT "$BASE/albums/$AID" ...  # second PUT must succeed (idempotent)
curl -s "$BASE/albums/00000000-0000-0000-0000-000000000000"  # must return 404

# S5: list
curl -s "$BASE/albums" | python3 -c "import sys,json;
print(len(json.load(sys.stdin)), 'albums')"

# S3: upload + poll to completed, verify seq != 0, url returns 200
curl -s -X POST "$BASE/albums/$AID/photos" -F "photo=@test.bin"
# ... poll status, verify seq and url

# S4/S7: delete + double-delete + verify 404
curl -X DELETE "$BASE/albums/$AID/photos/$PID"  # → 204
curl -s "$BASE/albums/$AID/photos/$PID"          # → {"error":"not found"}
curl -X DELETE "$BASE/albums/$AID/photos/$PID"  # → 204 (idempotent)

# S10: concurrent seq — all 5 must be unique
for i in $(seq 1 5); do
  curl -s -X POST "$BASE/albums/$AID/photos" -F "photo=@test.bin" &
done
wait  # verify all seqs are distinct, increasing
```

Only after all checks passed was a ChaosArena submission made.

---

15. If you had another week, what is the one thing you would change or add to your system to improve your score?

The system already achieves **190/190** — there is nothing to improve score-wise.

If the goal shifted to production readiness, the one highest-value addition would be **proper observability**: structured logging (zerolog), metrics (Prometheus), and distributed tracing (OpenTelemetry). The bugs encountered during this project (seq=0, S9 race, GET /albums timeout) were diagnosed through ad-hoc `curl` testing and reading service logs via SSM. With proper metrics, these would have been visible as anomalies before submission:

- `seq_assignment_latency_ms` histogram: a spike would flag the Redis connectivity issue

- `http_handler_duration_seconds{route="/albums"}` histogram: a 30-second tail would flag the index bug
- `worker_upload_duration_seconds` histogram: separate buckets for small vs large files

This is the kind of investment that pays off continuously in production systems.

---

16. How did you add value over and above what Claude could do in this assignment?

Claude provided the implementation scaffolding quickly, but the architectural decisions, real-time diagnosis, and strategic direction came from me:

Architecture decisions:

- Identified that co-location with ChaosArena (same AWS region) matters more than raw compute power — a local Mac M3 Pro would score worse than a t2.micro in us-west-2 because cross-country network latency adds 60ms per request and reduces S3 upload throughput 10-50×.
- Directed the upgrade path: t3.large (burst credits) → m5.large (10 Gbps) → 4× c5n.large (25 Gbps each). Claude knew the instance types but the sequencing and reasoning was mine.
- Recognized from my friend's architecture diagram that **c5n.large** (network-optimized) was the key differentiator, not c5.large or m5.large. The `c5n` prefix specifically means higher network bandwidth.

Real-time debugging:

- Identified t3.large burst credit exhaustion from CloudWatch metrics (13 credits out of 288) as the root cause of S15 scoring 0. Claude suggested the mechanism but I debugged the specific evidence.
- Diagnosed the GET /albums 30-second timeout as a UUID-to-text cast index problem by directly querying the DB (`SELECT COUNT(*) FROM albums` returned 260,686) and cross-referencing with the `ss -tlnp` and `journalctl` outputs.
- Identified the Redis bind issue (instances using `sg-0f6369c91322c7ba7` but Redis still bound to `127.0.0.1`) and directed the switch to ElastiCache as a more reliable solution.

Strategic decisions:

- Recognized when to stop optimizing (the pipe streaming approach failed twice — I stopped and redirected to the proven ParseMultipartForm approach).
- Decided to match the friend's exact architecture from the diagram rather than continuing to iterate on our own approach. Looking at the evidence and pivoting quickly was the right call.
- Managed the lab budget: understood which instance types were within the IAM policy restrictions (c5n.large was allowed, c5.xlarge was not), preventing wasted attempts.

Scope control:

- Claude's tendency is to add abstractions. I consistently directed it to keep things simple — no ORM, no message broker, no complex load balancer config, minimal dependencies. The architecture is intentionally lean because every added component adds latency.